```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
char stack[100];
int top = -1, size;
int flag=0;
// Function to push an item onto the stack
void push(char item) {
        if (top >= size - 1) {
                printf("\nStack Overflow.");
      exit(1);
   } else {
      stack[++top] = item;
   }
}
// Function to pop an item from the stack
char pop() {
   if (top < 0) {
      printf("\nStack Underflow\n");
      exit(1);
   } else {
      return stack[top--];
   }
}
// Function to check if a character is an operator
int is_operator(char symbol) {
   return (symbol == '^' || symbol == '*' || symbol == '/' || symbol == '+' ||
symbol == '-');
}
// Function to return the precedence of operators
int precedence(char symbol) {
   if (symbol == '^') return 3; // Highest precedence
   if (symbol == '*' || symbol == '/') return 2;
   if (symbol == '+' || symbol == '-') return 1;
   return 0; // Non-operator
}
// Function to convert infix to postfix
void InfixToPostfix(char infix_exp[], char postfix_exp[]) {
   int i = 0, j = 0;
   char item;
   push('(');
   strcat(infix_exp, ")");
   item = infix_exp[i];
   while (item != '\0') {
      if (item == '(') {
         push(item);
      }else if (isdigit(item){
```

```
        flag=1;
        postfix_exp[j++] = item; // Append operand to postfix
    }else if(isalpha(item)){
     postfix_exp[j++]=item;
     }
     else if (is_operator(item)) {
       while (top != -1 && is_operator(stack[top]) &&
              precedence(stack[top]) >= precedence(item)) {
          postfix_exp[j++] = pop(); // Pop operator to postfix
       }
       push(item); // Push current operator
    } else if (item == ')') {
       while (top != -1 && (stack[top] != '(')) {
          postfix_exp[j++] = pop(); // Pop until '('
       }
       pop(); // Remove '(' from stack
    } else {
       printf("\nInvalid infix Expression.\n");
       exit(1);
    }
    i++;
    item = infix_exp[i];
}
postfix_exp[j] = '\0'; // Null terminate postfix expression
}
// Function to evaluate a postfix expression
int evaluatePostfix(char postfix_exp[]) {
    int stack[100];
    int top = -1; // Reset top for evaluation
    int i = 0, operand1, operand2, result;
    while (postfix_exp[i] != '\0') {
       if (isdigit(postfix_exp[i])) {
          // Convert character to integer and push to stack
          stack[++top] = postfix_exp[i] - '0';
       } else if (is_operator(postfix_exp[i])) {
          operand2 = stack[top--]; // Pop second operand
          operand1 = stack[top--]; // Pop first operand
          switch (postfix_exp[i]) {
             case '+':
                result = operand1 + operand2;
                break;
             case '-':
                result = operand1 - operand2;
                break;
             case '*':
                result = operand1 * operand2;
                break;
             case '/':
```

```
            if (operand2 == 0) {
                printf("Division by zero error\n");
                exit(1);
            }
            result = operand1 / operand2;
            break;
        case '^':
            result = 1;
            for (int j = 0; j < operand2; j++) {
                result *= operand1;
            }
            break;
        default:
            printf("\nInvalid operator: %c\n", postfix_exp[i]);
            exit(1);
    }
    stack[++top] = result; // Push result back onto stack
  } else {
    printf("\nInvalid character in postfix expression.\n");
    exit(1);
  }
  i++;
}
return stack[top]; // Result is the last element on the stack}
```

```
int main() {
  char infix[100], postfix[100];
  printf("\nEnter size of stack: ");
  scanf("%d", &size);
  printf("Assume the infix expression contains single letter variables
and single digit constants only.\n");
  printf("\nEnter Infix expression: ");
  scanf(" %s", infix);
 InfixToPostfix(infix, postfix);
 printf("Postfix Expression: %s\n", postfix);
  int result;
  if(flag==1) {
  result = evaluatePostfix(postfix);
  printf("Result of Postfix Evaluation: %d\n", result);
  }
  return 0;
}
```

OUTPUT

Enter size of stack: 10

Assume the infix expression contains single letter variables and single digit constants only.

Enter Infix expression: 1+9*7*8^2/5

Postfix Expression: 197*82^*5/+

Result of Postfix Evaluation: 807

Enter size of stack: 10

Assume the infix expression contains single letter variables and single digit constants only.

Enter Infix expression: a+b*c/d+e

Postfix Expression: abc*d/+e+

Enter size of stack: 10

Assume the infix expression contains single letter variables and single digit constants only.

Enter Infix expression: 1+3*7*9/6^2+9*4

Postfix Expression: 137*9*62^/+94*+

Result of Postfix Evaluation: 42